# PYTHON CASES

Code • Output • Algorithm • Pseudocode • Flowchart

Prepared from the 10 Python case screenshots provided. Each case includes the program, sample run, step-by-step algorithm, pseudocode, and a text flowchart.

## Contents


1. Case 1: Attendance Eligibility
2. Case 2: Student Marks and Grade
3. Case 3: Compare Vehicle Fuel Costs
4. Case 4: Electricity Bill
5. Case 5: Event Budget
6. Case 6: Bank Deposit and Withdrawal
7. Case 7: Shopping Bill and Discount
8. Case 8: Examination Marks Trend
9. Case 9: Water Tank Level for Three Days
10. Case 10: Library Overdue Fine

# Case 1: Attendance Eligibility

## 1. Python Code

```
attended = int(input("Enter attended classes: "))
total = int(input("Enter total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = attended / total * 100
    print("Attendance percentage:", round(percentage, 2), "%")

    if percentage >= 75:
        print("Status: Eligible")
    else:
        extra = 0
        while (attended + extra) / (total + extra) < 0.75:
            extra += 1

        print("Status: Not eligible yet")
        print("Additional classes required:", extra)
```

## 2. Sample Output

```
Enter attended classes: 10
Enter total classes: 15
Attendance percentage: 66.67 %
Status: Not eligible yet
Additional classes required: 5
```

## 3. Algorithm / Standard Flow

11. Start.
12. Read attended classes and total classes.
13. Validate that total is positive and attended is between zero and total.
14. If invalid, display an error; otherwise calculate and display attendance percentage.
15. If percentage is at least 75%, display eligible.
16. Otherwise, count additional attended classes until the percentage reaches 75%, then display the count.
17. Stop.

## 4. Pseudocode

```
START
 INPUT attended, total
 IF total <= 0 OR attended < 0 OR attended > total THEN
    PRINT "Invalid input"
 ELSE
    percentage ← attended / total × 100
    PRINT percentage
    IF percentage >= 75 THEN
       PRINT "Eligible"
    ELSE
       extra ← 0
```

```
        WHILE (attended + extra) / (total + extra) < 0.75
            extra ← extra + 1
        END WHILE
        PRINT "Not eligible yet"
        PRINT extra
    END IF
 END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START]
   |
[Input attended, total]
   |
{total <= 0 OR attended < 0 OR attended > total?}
   | Yes                    | No
[Print Invalid input]     [Calculate percentage]
   |                        |
  [END]                [Print percentage]
                           |
                  {percentage >= 75?}
                    | Yes      | No
              [Print Eligible] [extra = 0]
                    |           |
                  [END]   {attendance with extra < 75%?}
                              | Yes
                          [extra = extra + 1]
                              |
                          (repeat test)
                              |
                             No
                  [Print Not eligible + extra]
                              |
                            [END]
```

# Case 2: Student Marks and Grade

## 1. Python Code

```
marks = []

for i in range(5):
    mark = int(input("Enter subject mark: "))
    marks.append(mark)

valid = True

for mark in marks:
    if mark < 0 or mark > 100:
        valid = False

if not valid:
    print("Invalid marks")
else:
    total = sum(marks)
    percentage = total / 5

    print("Total marks:", total)
    print("Percentage:", percentage)

    if min(marks) < 35:
        print("Result: Fail")
    else:
        print("Result: Pass")

    if percentage >= 90:
        grade = "A"
    elif percentage >= 75:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
    elif percentage >= 35:
        grade = "D"
    else:
        grade = "F"

    print("Grade:", grade)
```

## 2. Sample Output

```
Enter subject mark: 56
Enter subject mark: 87
Enter subject mark: 90
Enter subject mark: 87
Enter subject mark: 65
Total marks: 385
Percentage: 77.0
```

```
Result: Pass
Grade: B
```

## 3. Algorithm / Standard Flow

18. Start.
19. Read five subject marks.
20. Check that every mark is from 0 through 100.
21. If invalid, display an error.
22. Otherwise calculate and display total and percentage.
23. Fail if any subject mark is below 35; otherwise pass.
24. Assign grade from percentage and display it.
25. Stop.

## 4. Pseudocode

```
START
  marks ← empty list
  REPEAT 5 times
     INPUT mark
     ADD mark to marks
  END REPEAT
  valid ← TRUE
  FOR each mark in marks
     IF mark < 0 OR mark > 100 THEN
        valid ← FALSE
     END IF
  END FOR
  IF valid = FALSE THEN
     PRINT "Invalid marks"
  ELSE
     total ← SUM(marks)
     percentage ← total / 5
     PRINT total, percentage
     IF MIN(marks) < 35 THEN
        PRINT "Fail"
     ELSE
        PRINT "Pass"
     END IF
     IF percentage >= 90 THEN grade ← "A"
     ELSE IF percentage >= 75 THEN grade ← "B"
     ELSE IF percentage >= 60 THEN grade ← "C"
     ELSE IF percentage >= 35 THEN grade ← "D"
     ELSE grade ← "F"
     END IF
     PRINT grade
  END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [Input 5 marks] → {Any mark < 0 or > 100?}
                                  | Yes              | No
                          [Print Invalid]     [Calculate total & %]
```


[END]
[Any mark < 35?]
Yes
No
[Print Fail]
[Print Pass]
[Assign grade
from percentage]
[Print grade]
[END]

# Case 3: Compare Vehicle Fuel Costs

## 1. Python Code

```
distance = float(input("Enter distance in km: "))
mileage1 = float(input("Enter Vehicle 1 mileage: "))
mileage2 = float(input("Enter Vehicle 2 mileage: "))
fuel_price = float(input("Enter fuel price per litre: "))

if distance <= 0 or mileage1 <= 0 or mileage2 <= 0 or fuel_price < 0:
    print("Invalid input")
else:
    fuel1 = distance / mileage1
    fuel2 = distance / mileage2

    cost1 = fuel1 * fuel_price
    cost2 = fuel2 * fuel_price

    print("Vehicle 1 fuel:", round(fuel1, 2), "litres")
    print("Vehicle 1 cost: Rs.", round(cost1, 2))
    print("Vehicle 2 fuel:", round(fuel2, 2), "litres")
    print("Vehicle 2 cost: Rs.", round(cost2, 2))

    if cost1 < cost2:
        print("Vehicle 1 is cheaper")
    elif cost2 < cost1:
        print("Vehicle 2 is cheaper")
    else:
        print("Both vehicles cost the same")
```

## 2. Sample Output

```
Enter distance in km: 45
Enter Vehicle 1 mileage: 48
Enter Vehicle 2 mileage: 45
Enter fuel price per litre: 110
Vehicle 1 fuel: 0.94 litres
Vehicle 1 cost: Rs. 103.12
Vehicle 2 fuel: 1.0 litres
Vehicle 2 cost: Rs. 110.0
Vehicle 1 is cheaper
```

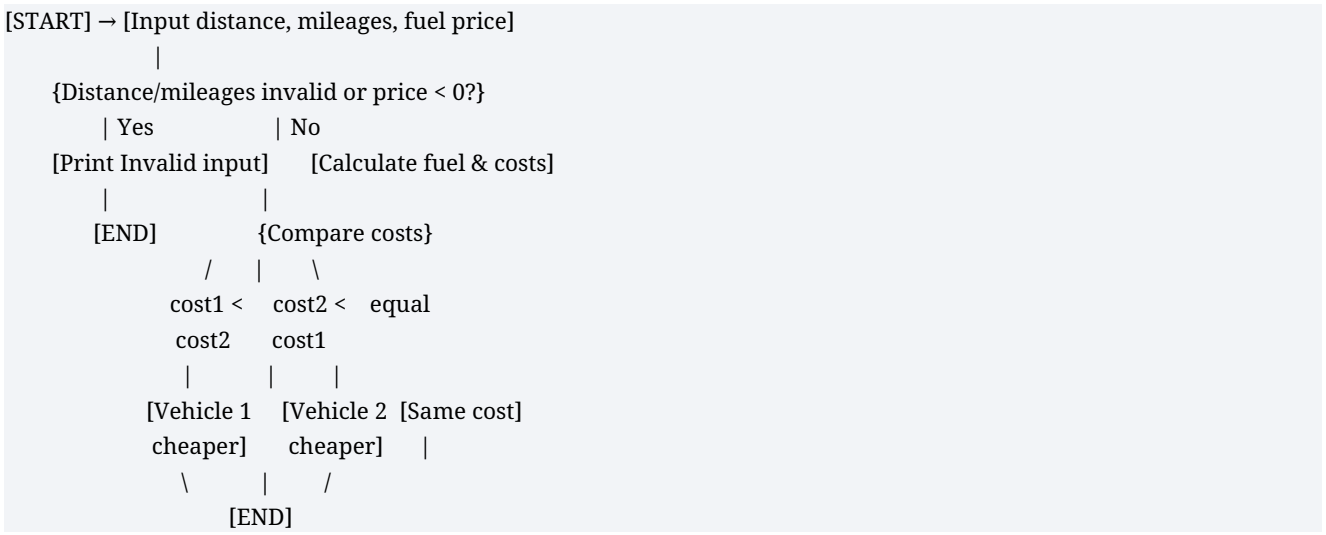
## 3. Algorithm / Standard Flow

26. Start.
27. Read distance, both vehicle mileages, and fuel price.
28. Validate the inputs.
29. Calculate fuel needed and cost for each vehicle.
30. Display fuel and cost for both vehicles.
31. Compare costs and display the cheaper vehicle, or a tie.
32. Stop.

## 4. Pseudocode

```
START
  INPUT distance, mileage1, mileage2, fuel_price
  IF distance <= 0 OR mileage1 <= 0 OR mileage2 <= 0 OR fuel_price < 0 THEN
     PRINT "Invalid input"
  ELSE
     fuel1 ← distance / mileage1
     fuel2 ← distance / mileage2
     cost1 ← fuel1 × fuel_price
     cost2 ← fuel2 × fuel_price
     PRINT fuel1, cost1, fuel2, cost2
     IF cost1 < cost2 THEN
        PRINT "Vehicle 1 is cheaper"
     ELSE IF cost2 < cost1 THEN
        PRINT "Vehicle 2 is cheaper"
     ELSE
        PRINT "Both vehicles cost the same"
     END IF
  END IF
STOP
```

## 5. Flowchart (Text Representation)



# Case 4: Electricity Bill

## 1. Python Code

```
units = int(input("Enter electricity units: "))

if units < 0:
   print("Invalid units")
else:
   if units <= 100:
      bill = units * 2
   elif units <= 200:
      bill = 200 + (units - 100) * 3
   else:
      bill = 500 + (units - 200) * 5

   bill = bill + 50

   print("Electricity units:", units)
   print("Fixed charge: Rs. 50")
   print("Total bill: Rs.", bill)
```

## 2. Sample Output

```
Enter electricity units: 53
Electricity units: 53
Fixed charge: Rs. 50
Total bill: Rs. 156
```

## 3. Algorithm / Standard Flow

33. Start.
34. Read electricity units.
35. If units are negative, display invalid input.
36. Otherwise calculate the slab charge: up to 100 units at Rs. 2/unit; 101–200 units with the first Rs. 200 charge plus Rs. 3/unit above 100; above 200 units with Rs. 500 plus Rs. 5/unit above 200.
37. Add fixed charge of Rs. 50.
38. Display units and total bill.
39. Stop.

## 4. Pseudocode

```
START
 INPUT units
 IF units < 0 THEN
    PRINT "Invalid units"
 ELSE
    IF units <= 100 THEN
       bill ← units × 2
    ELSE IF units <= 200 THEN
       bill ← 200 + (units - 100) × 3
    ELSE
       bill ← 500 + (units - 200) × 5
    END IF
```

```
    bill ← bill + 50
    PRINT units, "Fixed charge: Rs. 50", bill
  END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [Input units] → {units < 0?}
                          | Yes          | No
                  [Print Invalid]  {units <= 100?}
                       |          | Yes       | No
                    [END]    [bill=u×2] {units <= 200?}
                                  | Yes       | No
                              [bill=200+...] [bill=500+...]
                                  \          /
                                   [Add Rs. 50]
                                       |
                                   [Print bill]
                                       |
                                     [END]
```

# Case 5: Event Budget

## 1. Python Code

```
venue = float(input("Enter venue expense: "))
equipment = float(input("Enter equipment expense: "))
catering = float(input("Enter catering expense: "))
transport = float(input("Enter transport expense: "))

participants = int(input("Enter number of participants: "))
fee = float(input("Enter registration fee per person: "))
sponsorship = float(input("Enter sponsorship amount: "))

if min(venue, equipment, catering, transport, fee, sponsorship) < 0 or participants < 0:
    print("Invalid input")
else:
    expenses = venue + equipment + catering + transport
    income = participants * fee + sponsorship
    balance = income - expenses

    print("Total expenses: Rs.", expenses)
    print("Total income: Rs.", income)

    if balance >= 0:
        print("Surplus: Rs.", balance)
    else:
        print("Shortage: Rs.", abs(balance))
```

## 2. Sample Output

```
Enter venue expense: 10000
Enter equipment expense: 200
Enter catering expense: 2000
Enter transport expense: 270
Enter number of participants: 45
Enter registration fee per person: 150
Enter sponsorship amount: 1600
Total expenses: Rs. 12470.0
Total income: Rs. 8350.0
Shortage: Rs. 4120.0
```

## 3. Algorithm / Standard Flow

40. Start.
41. Read event expenses, participant count, fee, and sponsorship.
42. Validate that inputs are non-negative.
43. Calculate total expenses and total income.
44. Calculate balance as income minus expenses.
45. Display surplus if balance is non-negative; otherwise display shortage.
46. Stop.

## 4. Pseudocode

```
START
  INPUT venue, equipment, catering, transport
  INPUT participants, fee, sponsorship
  IF any cost, fee or sponsorship < 0 OR participants < 0 THEN
     PRINT "Invalid input"
  ELSE
     expenses ← venue + equipment + catering + transport
     income ← participants × fee + sponsorship
     balance ← income - expenses
     PRINT expenses, income
     IF balance >= 0 THEN
        PRINT "Surplus", balance
     ELSE
        PRINT "Shortage", ABS(balance)
     END IF
  END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [Input expenses, participants, fee, sponsorship]
                    |
               {Any input < 0?}
                | Yes      | No
          [Print Invalid] [Calculate expenses & income]
               |               |
             [END]        [balance = income - expenses]
                                |
                         {balance >= 0?}
                          | Yes      | No
                    [Print Surplus] [Print Shortage]
                        \          /
                            [END]
```

# Case 6: Bank Deposit and Withdrawal

## 1. Python Code

```
balance = float(input("Enter opening balance: "))

if balance < 0:
   print("Invalid balance")
else:
   while True:
      print("\nD - Deposit")
      print("W - Withdraw")
      print("Q - Quit")

      choice = input("Enter choice: ").upper()

      if choice == "Q":
         break
      elif choice == "D" or choice == "W":
         amount = float(input("Enter amount: "))

         if amount <= 0:
            print("Enter a positive amount")
         elif choice == "D":
            balance += amount
            print("Deposit successful")
         elif amount > balance:
            print("Insufficient balance")
         else:
            balance -= amount
            print("Withdrawal successful")
      else:
         print("Invalid choice")

   print("Final balance: Rs.", balance)
```

## 2. Sample Output

```
Enter opening balance: 1000

D - Deposit
W - Withdraw
Q - Quit
Enter choice: D
Enter amount: 2000
Deposit successful

D - Deposit
W - Withdraw
Q - Quit
Enter choice: Q
Final balance: Rs. 3000.0
```

## 3. Algorithm / Standard Flow

47. Start.
48. Read opening balance and validate it.
49. Repeatedly show deposit, withdrawal, and quit options.
50. For deposit or withdrawal, read a positive amount.
51. Add deposits to balance; reject withdrawals larger than balance; otherwise subtract withdrawals.
52. Repeat until the user chooses quit.
53. Display final balance and stop.

## 4. Pseudocode

```
START
 INPUT balance
 IF balance < 0 THEN
    PRINT "Invalid balance"
 ELSE
    REPEAT
       DISPLAY menu: D, W, Q
       INPUT choice
       IF choice = "Q" THEN
          EXIT LOOP
       ELSE IF choice = "D" OR choice = "W" THEN
          INPUT amount
          IF amount <= 0 THEN
             PRINT "Enter a positive amount"
          ELSE IF choice = "D" THEN
             balance ← balance + amount
             PRINT "Deposit successful"
          ELSE IF amount > balance THEN
             PRINT "Insufficient balance"
          ELSE
             balance ← balance - amount
             PRINT "Withdrawal successful"
          END IF
       ELSE
          PRINT "Invalid choice"
       END IF
    UNTIL choice = "Q"
    PRINT balance
 END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [Input opening balance] → {balance < 0?}
                                  | Yes        | No
                        [Print invalid] [Show menu & input choice]
                             |                |
                          [END]          {choice = Q?}
                                       | Yes      | No
                                [Print balance] [D or W?]
                                       |          |
                                    [END]   [Input amount]
```

```
                    |
              {amount <= 0?}
              | Yes       | No
        [Print positive] {Deposit?}
                        | Yes       | No
                  [Add amount] {amount > balance?}
                        |       | Yes       | No
                      [Msg] [Insufficient] [Subtract amount]
                        \     |         /
                      [Show menu again]
```

# Case 7: Shopping Bill and Discount

## 1. Python Code

```
total = 0
valid = True

for i in range(3):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))

    if price < 0 or quantity < 0:
        valid = False
    else:
        cost = price * quantity
        total += cost
        print("Product cost: Rs.", cost)

if not valid:
    print("Invalid input; bill not generated")
else:
    if total >= 1000:
        discount = total * 0.10
    else:
        discount = 0

    final_bill = total - discount
    print("Subtotal: Rs.", total)
    print("Discount: Rs.", discount)
    print("Final bill: Rs.", final_bill)
```

## 2. Sample Output

```
Enter product price: 150
Enter quantity: 4
Product cost: Rs. 600.0
Enter product price: 200
Enter quantity: 3
Product cost: Rs. 600.0
Enter product price: 140
Enter quantity: 9
Product cost: Rs. 1260.0
Subtotal: Rs. 2460.0
Discount: Rs. 246.0
Final bill: Rs. 2214.0
```

## 3. Algorithm / Standard Flow

54. Start.
55. Initialize subtotal to zero and validity flag to true.
56. Repeat three times: read price and quantity; validate; calculate product cost and add it to subtotal.
57. If any input was invalid, do not generate the bill.
58. Otherwise apply a 10% discount when subtotal is at least Rs. 1000.
59. Display subtotal, discount, and final bill.

60. Stop.

## 4. Pseudocode

```
START
 total ← 0
 valid ← TRUE
 REPEAT 3 times
    INPUT price, quantity
    IF price < 0 OR quantity < 0 THEN
       valid ← FALSE
    ELSE
       cost ← price × quantity
       total ← total + cost
       PRINT cost
    END IF
 END REPEAT
 IF valid = FALSE THEN
    PRINT "Invalid input; bill not generated"
 ELSE
    IF total >= 1000 THEN discount ← total × 0.10
    ELSE discount ← 0
    END IF
    final_bill ← total - discount
    PRINT total, discount, final_bill
 END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [total=0, valid=TRUE] → [Input price & quantity]
                                        |
                                {price < 0 OR quantity < 0?}
                                   | Yes          | No
                               [valid=FALSE] [Calculate cost, add total]
                                   \             /
                                   {3 products entered?}
                                    | No
                               [Input next product]
                                    |
                                   Yes
                                    |
                                {valid = FALSE?}
                                  | Yes        | No
                              [Print invalid] {total >= 1000?}
                                  |            | Yes       | No
                                [END]    [10% discount] [No discount]
                                            \          /
                                          [Final bill]
                                              |
                                            [END]
```

# Case 8: Examination Marks Trend

## 1. Python Code

```
marks = []

for i in range(3):
    mark = float(input("Enter examination mark: "))
    marks.append(mark)

valid = True

for mark in marks:
    if mark < 0 or mark > 100:
        valid = False

if not valid:
    print("Invalid marks")
else:
    average = sum(marks) / len(marks)

    print("Examination marks:", marks)
    print("Average marks:", round(average, 2))

    if marks[2] > marks[0]:
        print("Performance: Improved")
    elif marks[2] < marks[0]:
        print("Performance: Declined")
    else:
        print("Performance: Unchanged")
```

## 2. Sample Output

```
Enter examination mark: 37
Enter examination mark: 56
Enter examination mark: 98
Examination marks: [37.0, 56.0, 98.0]
Average marks: 63.67
Performance: Improved
```

## 3. Algorithm / Standard Flow

61. Start.
62. Read three examination marks.
63. Validate every mark is within 0–100.
64. If invalid, display an error.
65. Otherwise calculate and display average.
66. Compare the third mark with the first and display improved, declined, or unchanged.
67. Stop.

## 4. Pseudocode

```
START
 marks ← empty list
```

```
REPEAT 3 times
   INPUT mark
   ADD mark to marks
END REPEAT
valid ← TRUE
FOR each mark in marks
   IF mark < 0 OR mark > 100 THEN valid ← FALSE
END FOR
IF valid = FALSE THEN
   PRINT "Invalid marks"
ELSE
   average ← SUM(marks) / 3
   PRINT marks, average
   IF third mark > first mark THEN
      PRINT "Improved"
   ELSE IF third mark < first mark THEN
      PRINT "Declined"
   ELSE
      PRINT "Unchanged"
   END IF
END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [Input 3 marks] → {Any mark outside 0–100?}
                              | Yes          | No
                    [Print Invalid] [Calculate & print average]
                          |                  |
                       [END]          {Third > first?}
                                    | Yes       | No
                              [Improved] {Third < first?}
                                          | Yes       | No
                                    [Declined] [Unchanged]
                                       \       /
                                         [END]
```

# Case 9: Water Tank Level for Three Days

## 1. Python Code

```
capacity = 1000
level = float(input("Enter initial water level: "))

if level < 0 or level > capacity:
    print("Invalid water level")
else:
    for day in range(1, 4):
        inflow = float(input("Enter inflow for day: "))
        outflow = float(input("Enter outflow for day: "))

        if inflow < 0 or outflow < 0:
            print("Invalid inflow or outflow")
            continue

        level = level + inflow - outflow

        if level > capacity:
            print("Overflow detected")
            level = capacity
        elif level < 0:
            print("Water shortage detected")
            level = 0

        print("Day", day, "water level:", level, "litres")
```

## 2. Sample Output

```
Enter initial water level: 400
Enter inflow for day: 290
Enter outflow for day: 250
Day 1 water level: 440.0 litres
Enter inflow for day: 124
Enter outflow for day: 168
Day 2 water level: 396.0 litres
Enter inflow for day: 130
Enter outflow for day: 87
Day 3 water level: 439.0 litres
```

## 3. Algorithm / Standard Flow

68. Start.
69. Set tank capacity to 1000 litres and read initial level.
70. Validate initial level is between zero and capacity.
71. For each of three days, read inflow and outflow.
72. If either is negative, report invalid data and continue to the next day.
73. Update level by adding inflow and subtracting outflow.
74. Clamp level to capacity on overflow or to zero on shortage; display the day's level.
75. Stop after day three.

## 4. Pseudocode

```
START
 capacity ← 1000
 INPUT level
 IF level < 0 OR level > capacity THEN
    PRINT "Invalid water level"
 ELSE
    FOR day ← 1 TO 3
       INPUT inflow, outflow
       IF inflow < 0 OR outflow < 0 THEN
          PRINT "Invalid inflow or outflow"
          CONTINUE to next day
       END IF
       level ← level + inflow - outflow
       IF level > capacity THEN
          PRINT "Overflow detected"
          level ← capacity
       ELSE IF level < 0 THEN
          PRINT "Water shortage detected"
          level ← 0
       END IF
       PRINT day, level
    END FOR
 END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [capacity=1000; input initial level]
                  |
          {level < 0 OR level > capacity?}
              | Yes          | No
          [Print Invalid] [Repeat for days 1 to 3]
              |               |
            [END]         [Input inflow, outflow]
                            |
                    {Either input < 0?}
                      | Yes        | No
                  [Print invalid] [Update level]
                      |             |
                  [Next day]     {level > capacity?}
                                  | Yes      | No
                              [Print overflow] {level < 0?}
                              [level=capacity] | Yes     | No
                                      [Print shortage] [Keep level]
                                      [level=0]        |
                                         \          /
                                       [Print day level]
                                            |
                                       [More days?]
                                        | Yes   | No
                                      [Repeat] [END]
```

# Case 10: Library Overdue Fine

## 1. Python Code

```
records = []
total_fine = 0

n = int(input("Enter number of books: "))

if n < 0:
   print("Invalid number of books")
else:
   valid = True

   for i in range(n):
      book_id = input("Enter book ID: ")
      days = int(input("Enter borrowing days: "))

      if days < 0:
         valid = False
      else:
         records.append((book_id, days))

   if not valid:
      print("Invalid borrowing duration")
   else:
      for book_id, days in records:
         overdue = max(0, days - 7)
         fine = overdue * 2
         total_fine += fine

         print("Book ID:", book_id)
         print("Overdue days:", overdue)
         print("Fine: Rs.", fine)

      print("Total fine: Rs.", total_fine)
```

## 2. Sample Output

```
Enter number of books: 3
Enter book ID: B101
Enter borrowing days: 7
Enter book ID: B102
Enter borrowing days: 10
Enter book ID: B103
Enter borrowing days: 12
Book ID: B101
Overdue days: 0
Fine: Rs. 0
Book ID: B102
Overdue days: 3
Fine: Rs. 6
Book ID: B103
```

```
Overdue days: 5
Fine: Rs. 10
Total fine: Rs. 16
```

## 3. Algorithm / Standard Flow

76. Start.
77. Read number of books and validate it.
78. For each book, read ID and borrowing duration; validate duration.
79. If any duration is negative, display an error.
80. Otherwise calculate overdue days as max(0, borrowing days minus 7).
81. Calculate each fine at Rs. 2 per overdue day and add it to total fine.
82. Display each book's fine and the total fine; stop.

## 4. Pseudocode

```
START
 records ← empty list
 total_fine ← 0
 INPUT n
 IF n < 0 THEN
    PRINT "Invalid number of books"
 ELSE
    valid ← TRUE
    REPEAT n times
       INPUT book_id, days
       IF days < 0 THEN valid ← FALSE
       ELSE ADD (book_id, days) to records
       END IF
    END REPEAT
    IF valid = FALSE THEN
       PRINT "Invalid borrowing duration"
    ELSE
       FOR each (book_id, days) in records
          overdue ← MAX(0, days - 7)
          fine ← overdue × 2
          total_fine ← total_fine + fine
          PRINT book_id, overdue, fine
       END FOR
       PRINT total_fine
    END IF
 END IF
STOP
```

## 5. Flowchart (Text Representation)

```
[START] → [total_fine=0; input n] → {n < 0?}
                                  | Yes       | No
                          [Print invalid] [Input each book ID & days]
                                  |               |
                                [END]      {Any days < 0?}
                                             | Yes      | No
                                     [Print invalid] [For each record]
                                             |            |
```

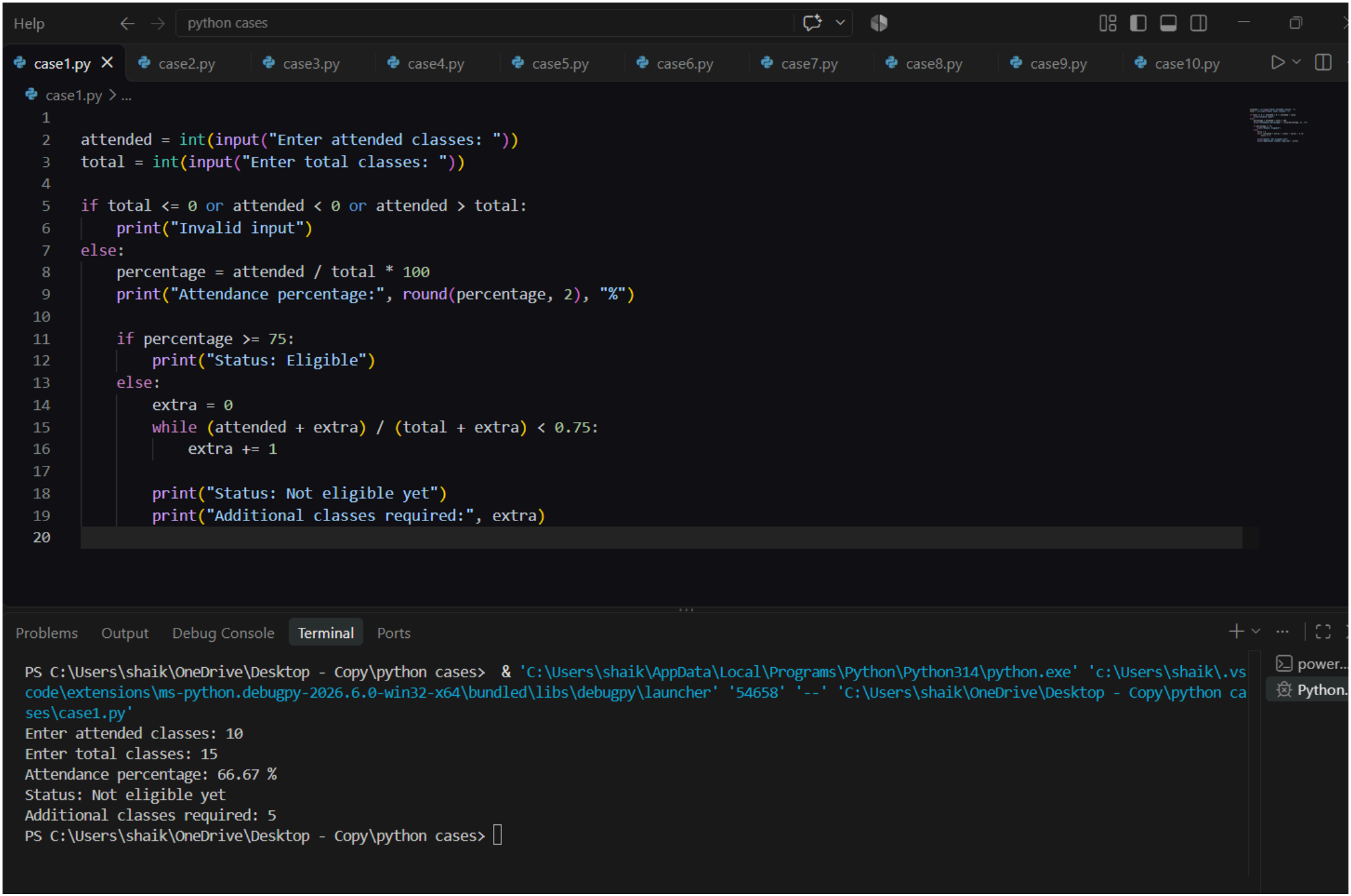
```
[END]    [overdue=max(0,days-7)]
              |
         [fine=overdue×2]
              |
         [Add to total]
              |
         [More records?]
          | Yes    | No
         [Repeat] [Print total]
                    |
                  [END]
```

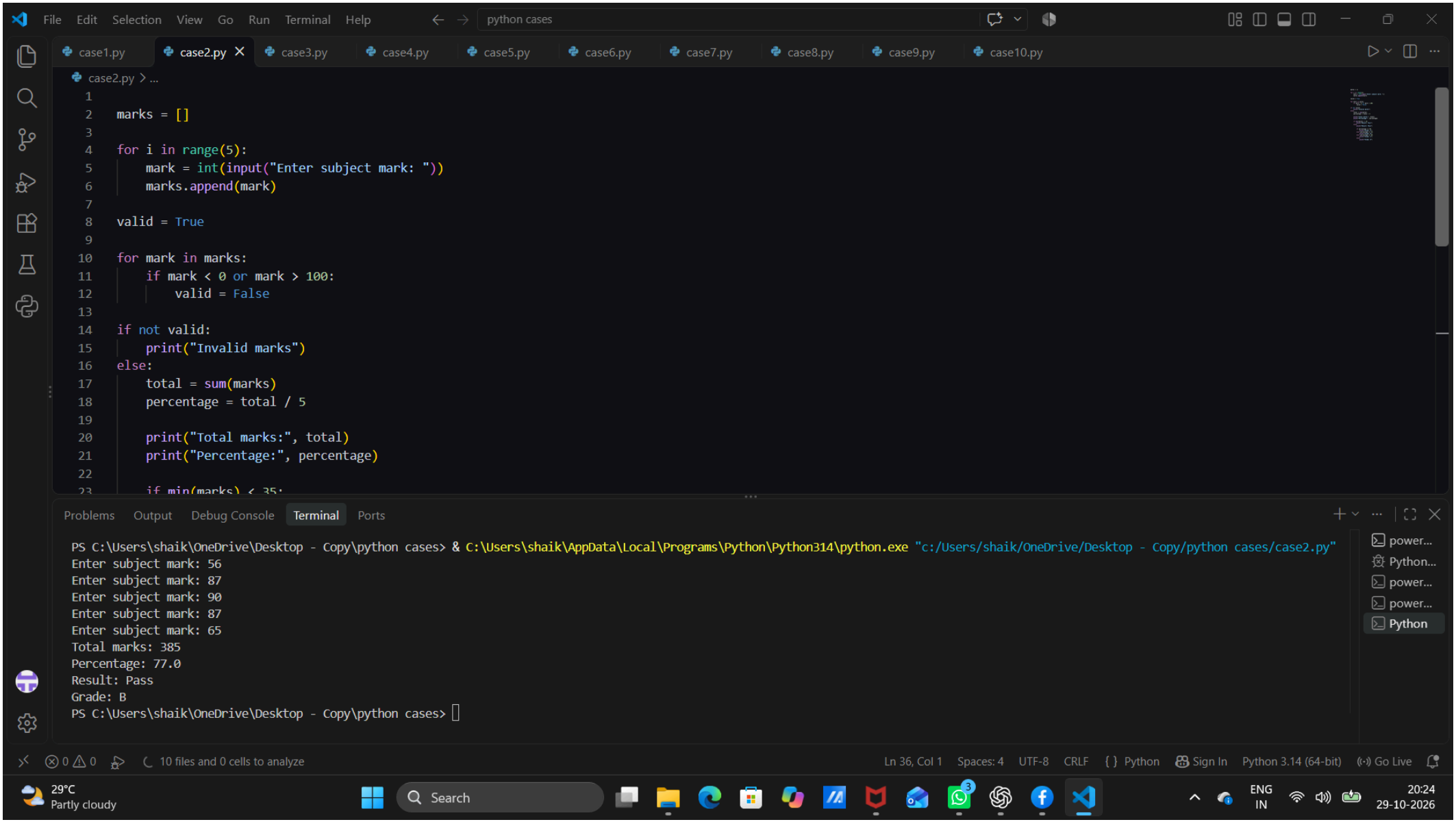
# Notes

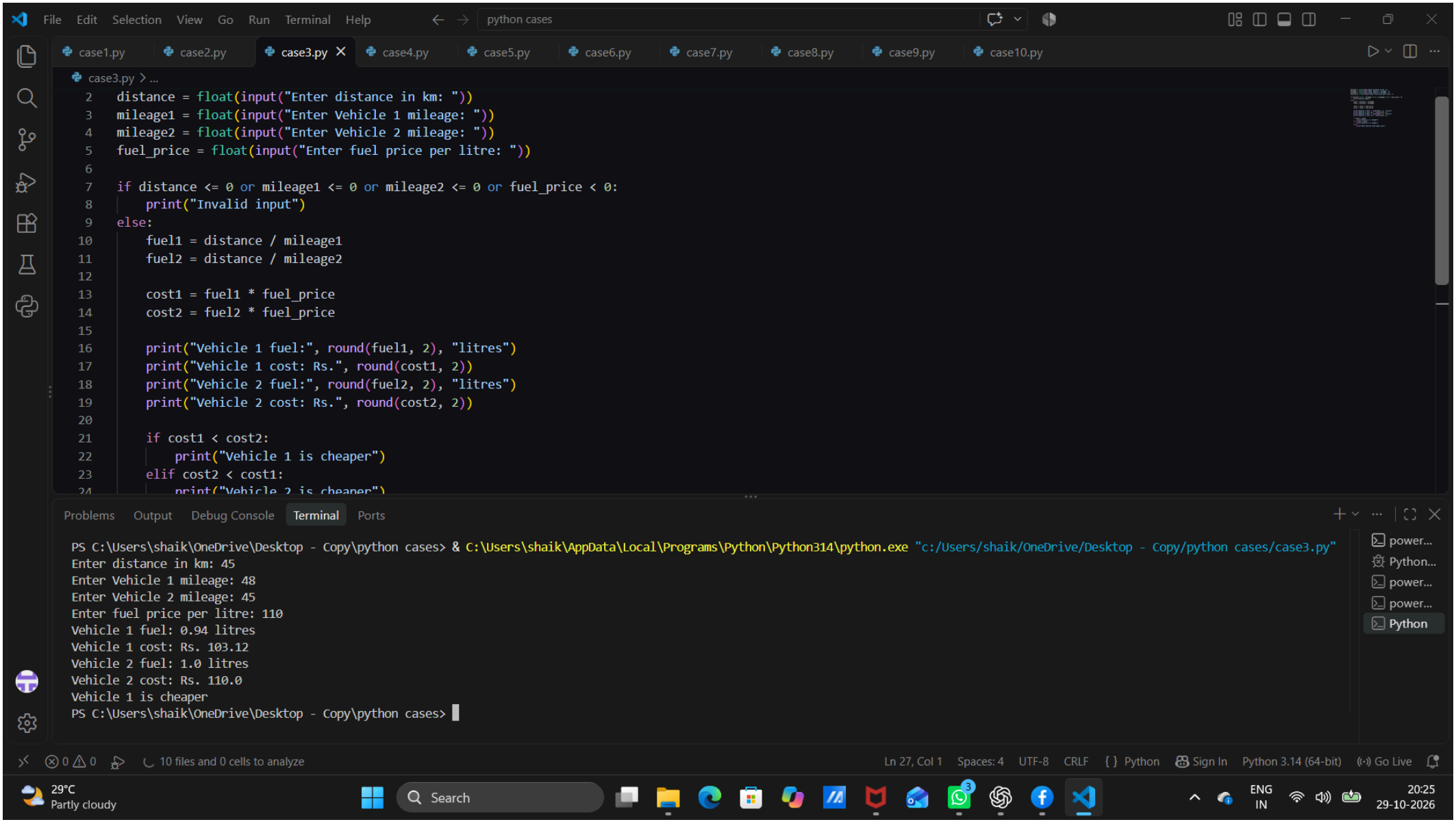
• The sample outputs are based on the values visible in the supplied screenshots.

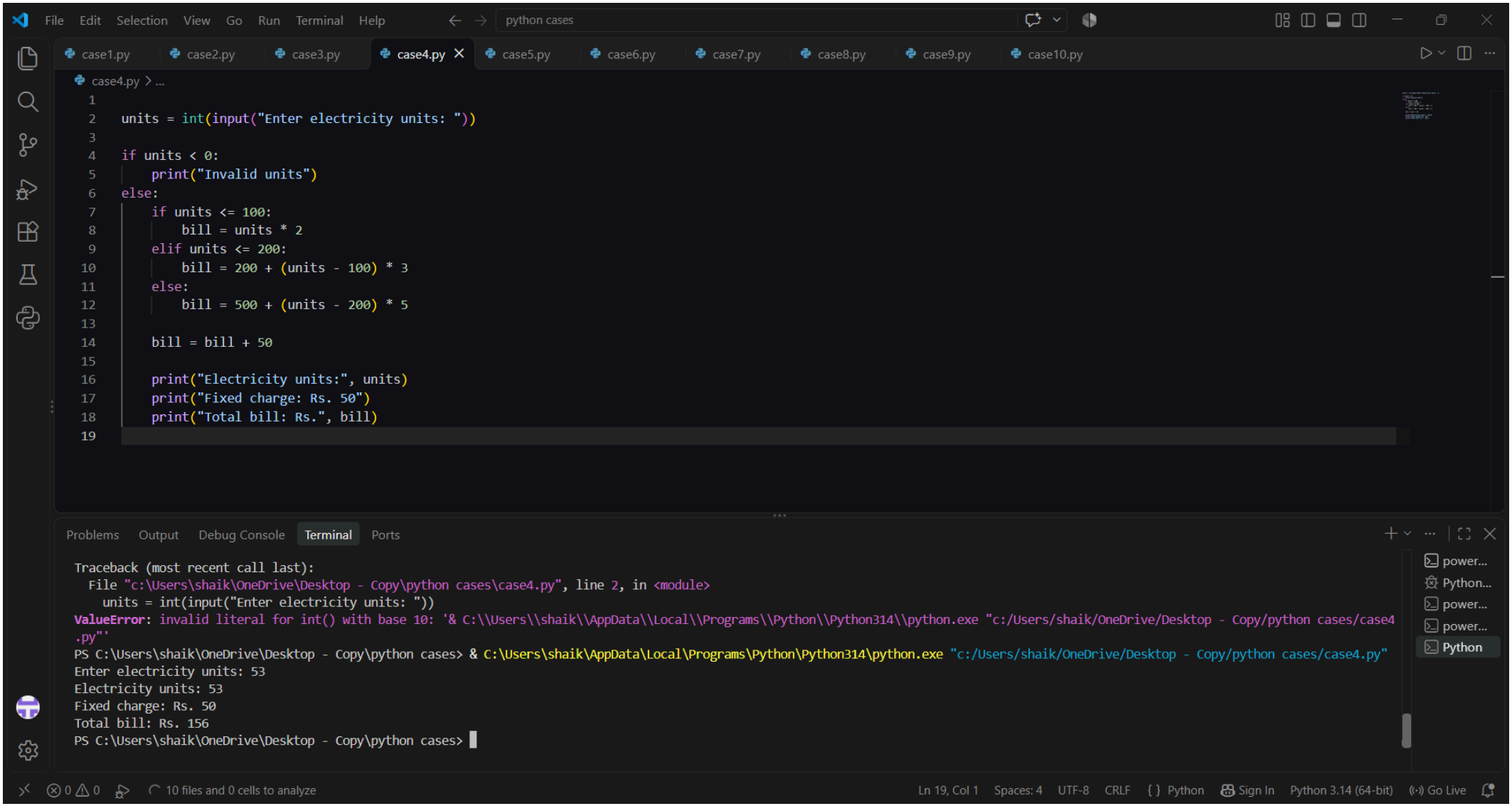
• Flowcharts are provided as text diagrams so they remain editable and easy to copy into a notebook. Use ovals for Start/Stop, parallelograms for Input/Output, rectangles for processing, and diamonds for decisions when drawing them graphically.

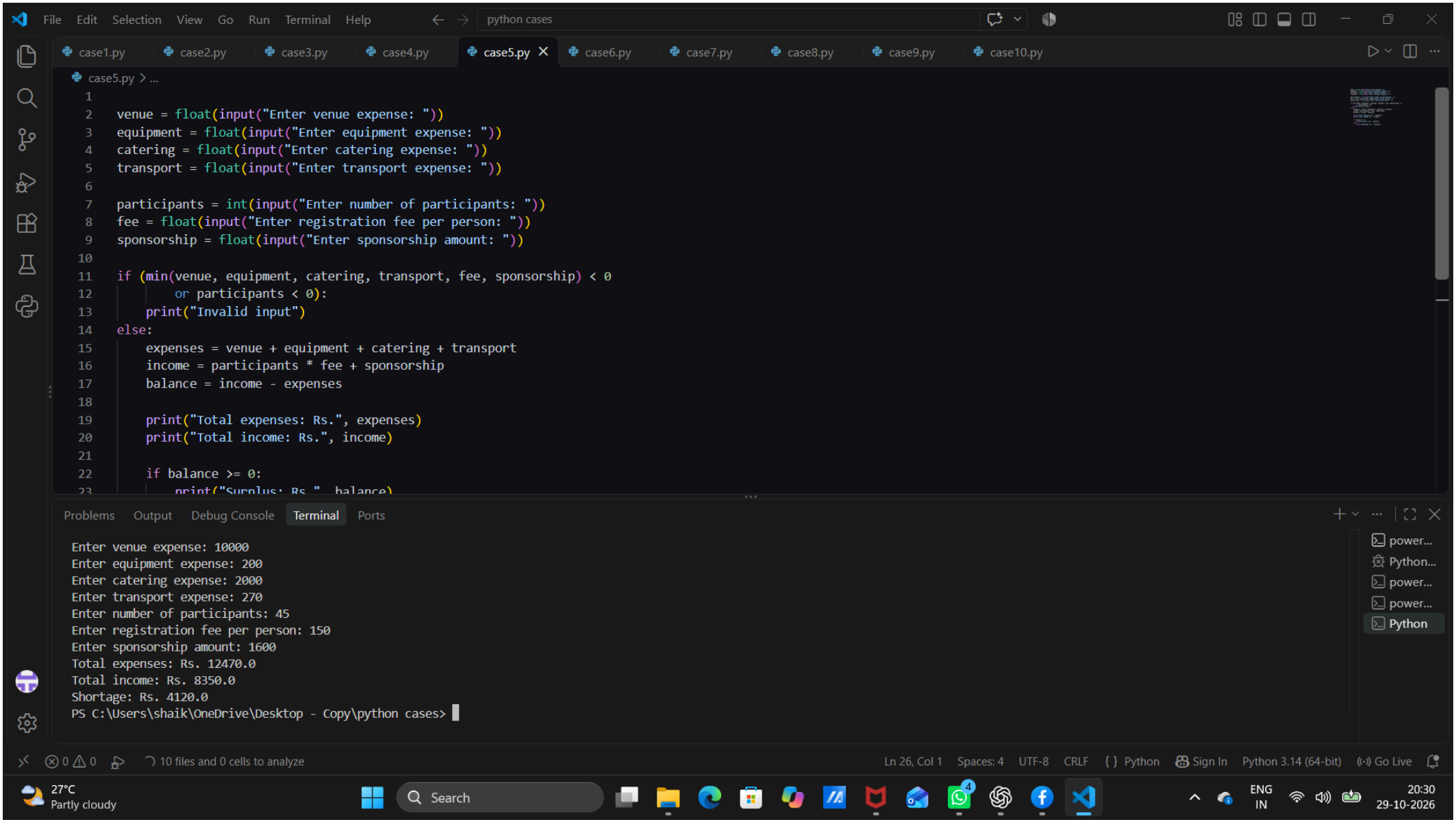
• Case 1 uses a semicolon-free Python if statement; write `if total <= 0 or attended < 0 or attended > total:` without a trailing semicolon.

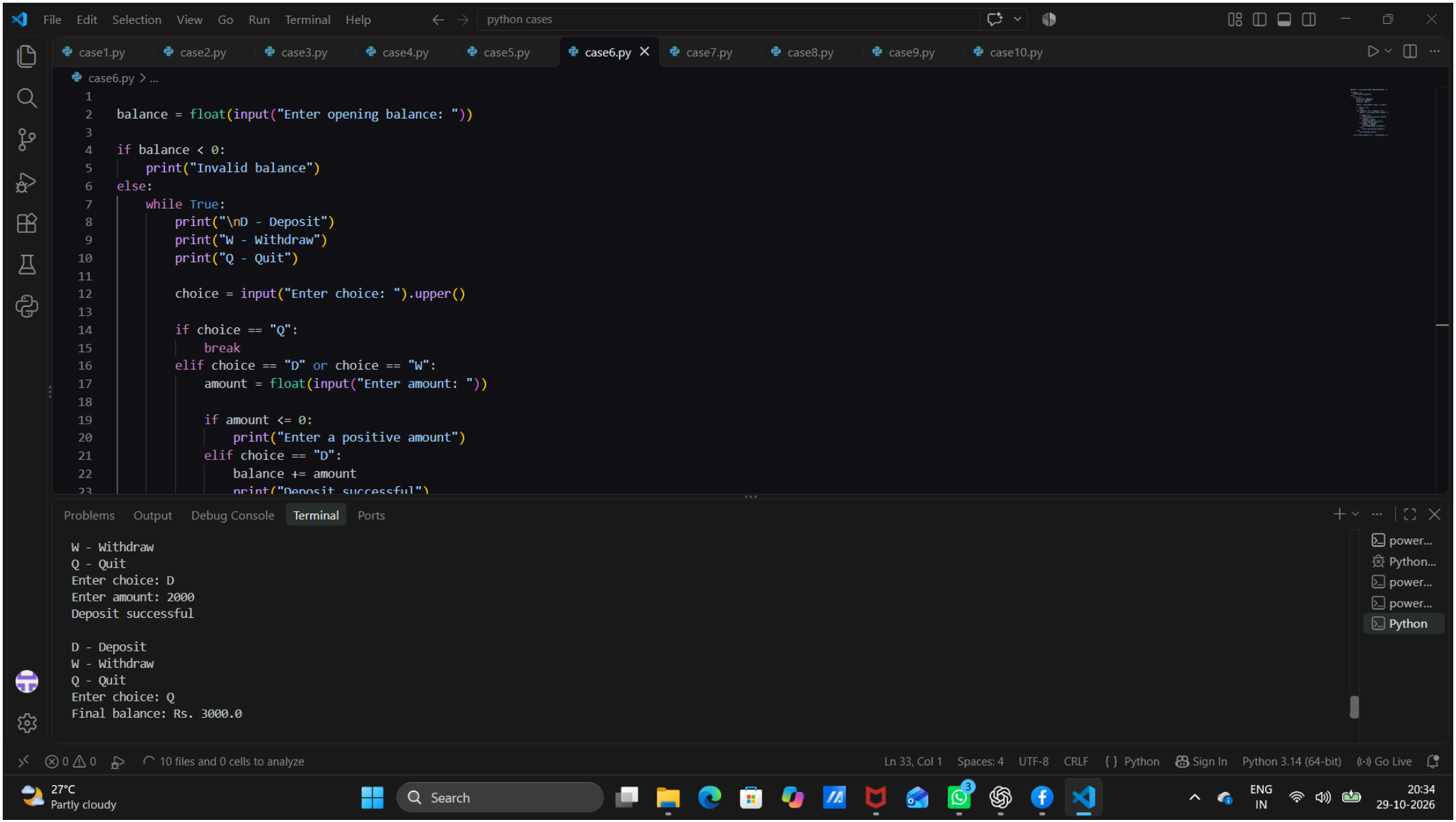

Help
python cases
case1.py
case2.py
case3.py
case4.py
case5.py
case6.py
case7.py
case8.py
case9.py
case10.py
case1.py > ...
attended = int(input("Enter attended classes: "))
total = int(input("Enter total classes: "))
if total <= 0 or attended < 0 or attended > total:
print("Invalid input")
else:
percentage = attended / total * 100
print("Attendance percentage:", round(percentage, 2), "%")
if percentage >= 75:
print("Status: Eligible")
else:
extra = 0
while (attended + extra) / (total + extra) < 0.75:
extra += 1
print("Status: Not eligible yet")
print("Additional classes required:", extra)
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & 'C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\shaik\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54658' '--' 'C:\Users\shaik\OneDrive\Desktop - Copy\python cases\case1.py'
Enter attended classes: 10
Enter total classes: 15
Attendance percentage: 66.67 %
Status: Not eligible yet
Additional classes required: 5
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>

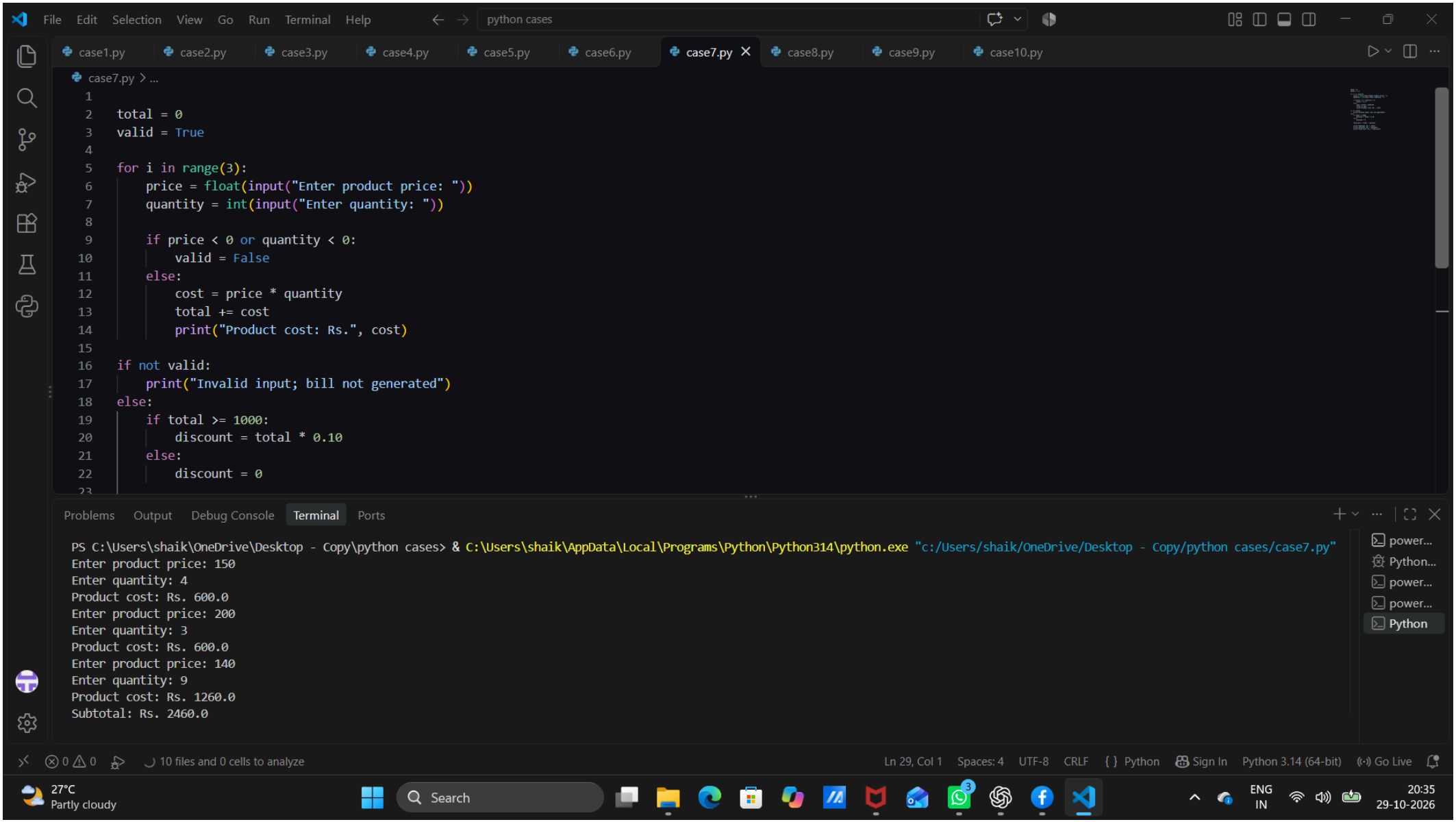

File Edit Selection View Go Run Terminal Help
python cases
case1.py
case2.py
case3.py
case4.py
case5.py
case6.py
case7.py
case8.py
case9.py
case10.py
case2.py > ...
1
2 marks = []
3
4 for i in range(5):
5 mark = int(input("Enter subject mark: "))
6 marks.append(mark)
7
8 valid = True
9
10 for mark in marks:
11 if mark < 0 or mark > 100:
12 valid = False
13
14 if not valid:
15 print("Invalid marks")
16 else:
17 total = sum(marks)
18 percentage = total / 5
19
20 print("Total marks:", total)
21 print("Percentage:", percentage)
22
Problems Output Debug Console Terminal Ports
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case2.py"
Enter subject mark: 56
Enter subject mark: 87
Enter subject mark: 90
Enter subject mark: 87
Enter subject mark: 65
Total marks: 385
Percentage: 77.0
Result: Pass
Grade: B
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 36, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live
29°C
Partly cloudy
Search
ENG
IN
20:24
29-10-2026

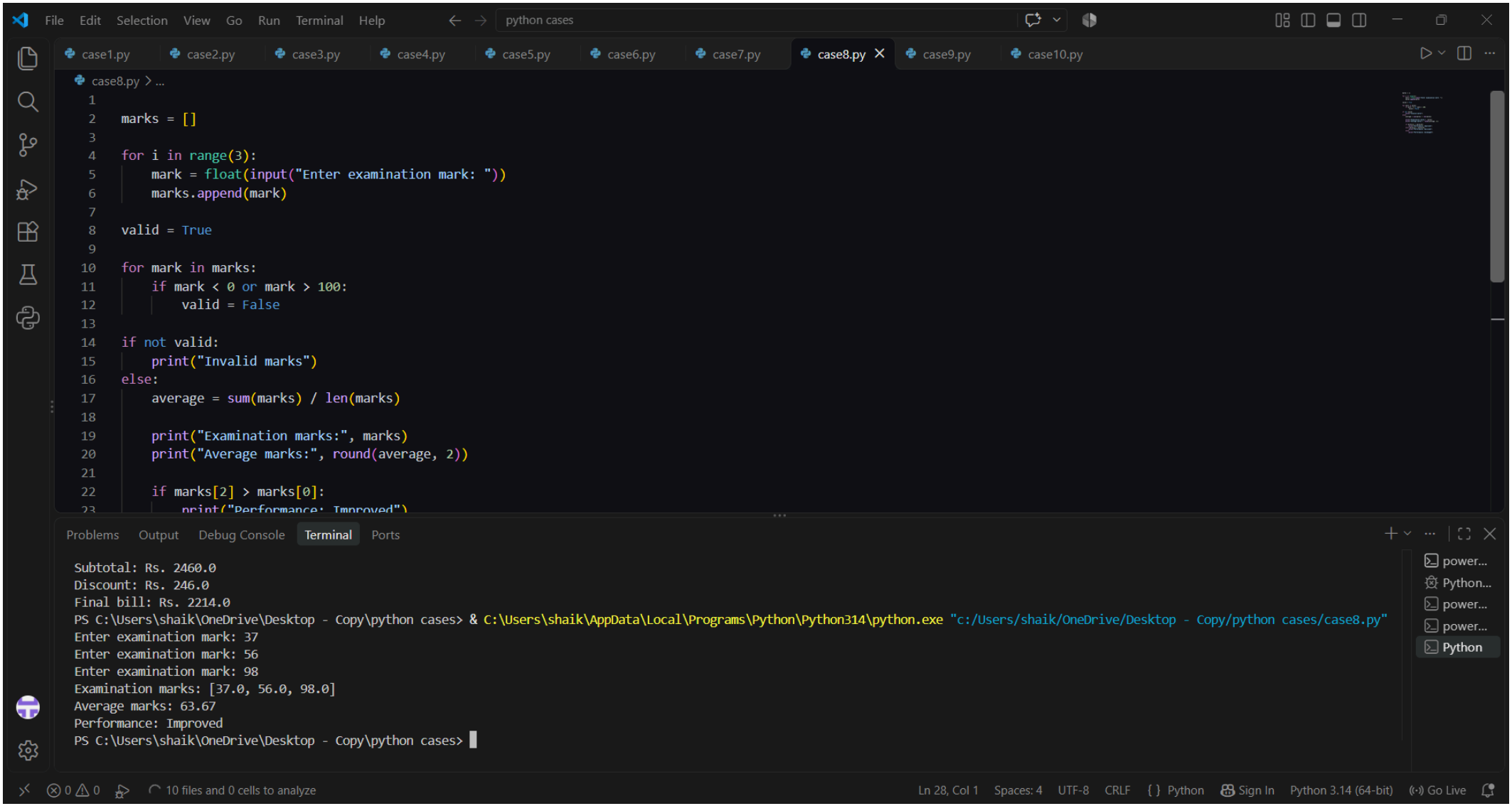

File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case3.py > ...
2 distance = float(input("Enter distance in km: "))
3 mileage1 = float(input("Enter Vehicle 1 mileage: "))
4 mileage2 = float(input("Enter Vehicle 2 mileage: "))
5 fuel_price = float(input("Enter fuel price per litre: "))
6
7 if distance <= 0 or mileage1 <= 0 or mileage2 <= 0 or fuel_price < 0:
8 print("Invalid input")
9 else:
10 fuel1 = distance / mileage1
11 fuel2 = distance / mileage2
12
13 cost1 = fuel1 * fuel_price
14 cost2 = fuel2 * fuel_price
15
16 print("Vehicle 1 fuel:", round(fuel1, 2), "litres")
17 print("Vehicle 1 cost: Rs.", round(cost1, 2))
18 print("Vehicle 2 fuel:", round(fuel2, 2), "litres")
19 print("Vehicle 2 cost: Rs.", round(cost2, 2))
20
21 if cost1 < cost2:
22 print("Vehicle 1 is cheaper")
23 elif cost2 < cost1:
Problems Output Debug Console Terminal Ports
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case3.py"
Enter distance in km: 45
Enter Vehicle 1 mileage: 48
Enter Vehicle 2 mileage: 45
Enter fuel price per litre: 110
Vehicle 1 fuel: 0.94 litres
Vehicle 1 cost: Rs. 103.12
Vehicle 2 fuel: 1.0 litres
Vehicle 2 cost: Rs. 110.0
Vehicle 1 is cheaper
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 27, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live
29°C
Partly cloudy
Search
ENG
IN
20:25
29-10-2026

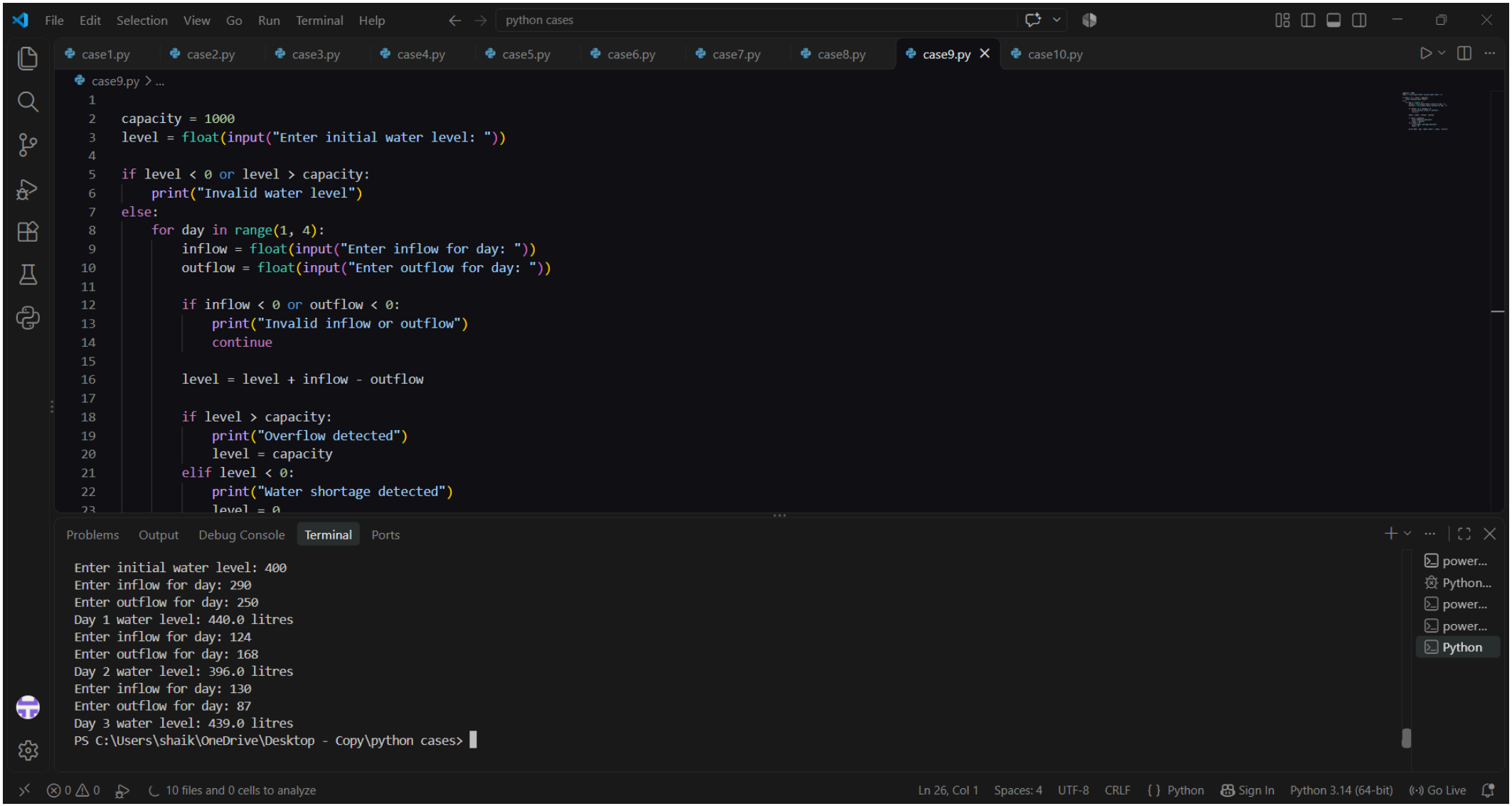

File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case4.py > ...
1
2 units = int(input("Enter electricity units: "))
3
4 if units < 0:
5 print("Invalid units")
6 else:
7 if units <= 100:
8 bill = units * 2
9 elif units <= 200:
10 bill = 200 + (units - 100) * 3
11 else:
12 bill = 500 + (units - 200) * 5
13
14 bill = bill + 50
15
16 print("Electricity units:", units)
17 print("Fixed charge: Rs. 50")
18 print("Total bill: Rs.", bill)
19
Problems Output Debug Console Terminal Ports
Traceback (most recent call last):
File "c:\Users\shaik\OneDrive\Desktop - Copy\python cases\case4.py", line 2, in <module>
units = int(input("Enter electricity units: "))
ValueError: invalid literal for int() with base 10: '& C:\\Users\\shaik\\AppData\\Local\\Programs\\Python\\Python314\\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case4.py"'
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case4.py"
Enter electricity units: 53
Electricity units: 53
Fixed charge: Rs. 50
Total bill: Rs. 156
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
0 0
10 files and 0 cells to analyze
Ln 19, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live

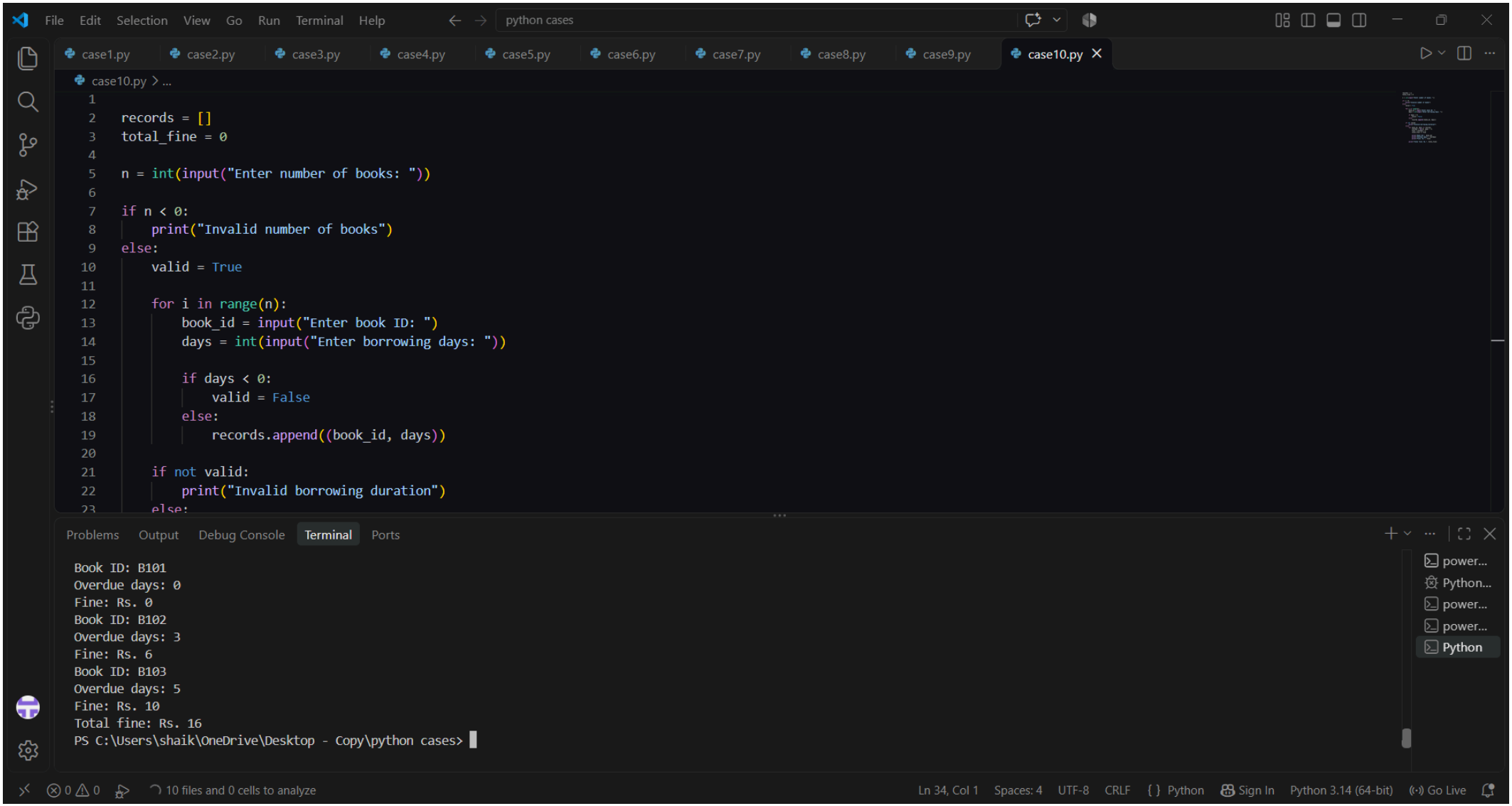

File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case5.py > ...
venue = float(input("Enter venue expense: "))
equipment = float(input("Enter equipment expense: "))
catering = float(input("Enter catering expense: "))
transport = float(input("Enter transport expense: "))
participants = int(input("Enter number of participants: "))
fee = float(input("Enter registration fee per person: "))
sponsorship = float(input("Enter sponsorship amount: "))
if (min(venue, equipment, catering, transport, fee, sponsorship) < 0
or participants < 0):
print("Invalid input")
else:
expenses = venue + equipment + catering + transport
income = participants * fee + sponsorship
balance = income - expenses
print("Total expenses: Rs.", expenses)
print("Total income: Rs.", income)
if balance >= 0:
Problems Output Debug Console Terminal Ports
Enter venue expense: 10000
Enter equipment expense: 200
Enter catering expense: 2000
Enter transport expense: 270
Enter number of participants: 45
Enter registration fee per person: 150
Enter sponsorship amount: 1600
Total expenses: Rs. 12470.0
Total income: Rs. 8350.0
Shortage: Rs. 4120.0
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 26, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live
27°C
Partly cloudy
Search
ENG
IN
20:30
29-10-2026


File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case6.py > ...
balance = float(input("Enter opening balance: "))
if balance < 0:
print("Invalid balance")
else:
while True:
print("\nD - Deposit")
print("W - Withdraw")
print("Q - Quit")
choice = input("Enter choice: ").upper()
if choice == "Q":
break
elif choice == "D" or choice == "W":
amount = float(input("Enter amount: "))
if amount <= 0:
print("Enter a positive amount")
elif choice == "D":
balance += amount
print("Deposit successful")
Problems Output Debug Console Terminal Ports
W - Withdraw
Q - Quit
Enter choice: D
Enter amount: 2000
Deposit successful
D - Deposit
W - Withdraw
Q - Quit
Enter choice: Q
Final balance: Rs. 3000.0
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 33, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live
27°C
Partly cloudy
Search
ENG
IN
20:34
29-10-2026


File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case7.py > ...
total = 0
valid = True
for i in range(3):
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))
if price < 0 or quantity < 0:
valid = False
else:
cost = price * quantity
total += cost
print("Product cost: Rs.", cost)
if not valid:
print("Invalid input; bill not generated")
else:
if total >= 1000:
discount = total * 0.10
else:
discount = 0
Problems Output Debug Console Terminal Ports
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case7.py"
Enter product price: 150
Enter quantity: 4
Product cost: Rs. 600.0
Enter product price: 200
Enter quantity: 3
Product cost: Rs. 600.0
Enter product price: 140
Enter quantity: 9
Product cost: Rs. 1260.0
Subtotal: Rs. 2460.0
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 29, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live
27°C
Partly cloudy
Search
ENG
IN
20:35
29-10-2026


File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case8.py > ...
marks = []
for i in range(3):
mark = float(input("Enter examination mark: "))
marks.append(mark)
valid = True
for mark in marks:
if mark < 0 or mark > 100:
valid = False
if not valid:
print("Invalid marks")
else:
average = sum(marks) / len(marks)
print("Examination marks:", marks)
print("Average marks:", round(average, 2))
if marks[2] > marks[0]:
Problems Output Debug Console Terminal Ports
Subtotal: Rs. 2460.0
Discount: Rs. 246.0
Final bill: Rs. 2214.0
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases> & C:\Users\shaik\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shaik/OneDrive/Desktop - Copy/python cases/case8.py"
Enter examination mark: 37
Enter examination mark: 56
Enter examination mark: 98
Examination marks: [37.0, 56.0, 98.0]
Average marks: 63.67
Performance: Improved
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 28, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case9.py > ...
capacity = 1000
level = float(input("Enter initial water level: "))
if level < 0 or level > capacity:
print("Invalid water level")
else:
for day in range(1, 4):
inflow = float(input("Enter inflow for day: "))
outflow = float(input("Enter outflow for day: "))
if inflow < 0 or outflow < 0:
print("Invalid inflow or outflow")
continue
level = level + inflow - outflow
if level > capacity:
print("Overflow detected")
level = capacity
elif level < 0:
print("Water shortage detected")
Problems Output Debug Console Terminal Ports
Enter initial water level: 400
Enter inflow for day: 290
Enter outflow for day: 250
Day 1 water level: 440.0 litres
Enter inflow for day: 124
Enter outflow for day: 168
Day 2 water level: 396.0 litres
Enter inflow for day: 130
Enter outflow for day: 87
Day 3 water level: 439.0 litres
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 26, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
python cases
case1.py case2.py case3.py case4.py case5.py case6.py case7.py case8.py case9.py case10.py
case10.py > ...
records = []
total_fine = 0
n = int(input("Enter number of books: "))
if n < 0:
print("Invalid number of books")
else:
valid = True
for i in range(n):
book_id = input("Enter book ID: ")
days = int(input("Enter borrowing days: "))
if days < 0:
valid = False
else:
records.append((book_id, days))
if not valid:
print("Invalid borrowing duration")
Problems Output Debug Console Terminal Ports
Book ID: B101
Overdue days: 0
Fine: Rs. 0
Book ID: B102
Overdue days: 3
Fine: Rs. 6
Book ID: B103
Overdue days: 5
Fine: Rs. 10
Total fine: Rs. 16
PS C:\Users\shaik\OneDrive\Desktop - Copy\python cases>
power...
Python...
power...
power...
Python
10 files and 0 cells to analyze
Ln 34, Col 1 Spaces: 4 UTF-8 CRLF { } Python Sign In Python 3.14 (64-bit) Go Live